IRIS FLOWER CLASSIFICATION

Student Name: Amirah Shayim Mohammad Alreshidi

ID:202310209

Machine learning(DSCE351)

PROJECT TYPE & CLASSES

This project focuses on applying Machine Learning techniques to build a model capable of classifying Iris flowers into their correct species.

The well-known Iris Dataset was chosen because it is widely used as an educational example in data science. It is simple in size but rich enough to demonstrate classification concepts effectively.

The project uses four main measurements of each flower as input features:

1. Sepal length

2. Sepal width

3. Petal length

4. Petal width

Type of Project: Classification – the task is to assign each flower to one of several predefined categories.

Number of Classes: 3

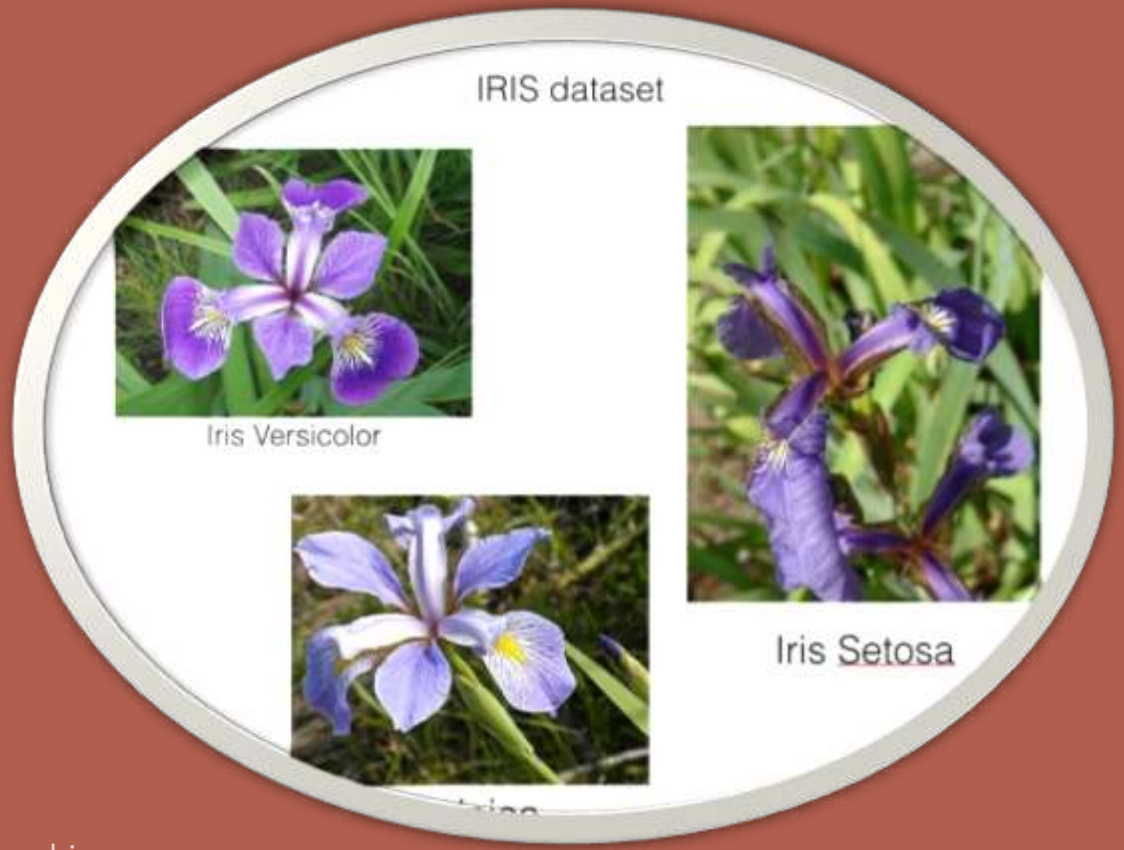
And The Classes are:

1. Setosa

2. Versicolor

3. Virginica

Importance of the Project: It provides a practical introduction to classification, showing how a machine can learn from simple numeric data to make accurate predictions.



DATASET DETAILS

records: 150 records each record represents anIris flower with 4 numerical features.

Features: Derived from physicalmeasurements of the flower, including:. Length and width of petals.. Length and width of sepals.

Classification: Includes a 'Species'column that indicates the type of Iris flower:·

- *Setosa*
- *Versicolor*
- *Virginica*

These features provide quantitative values that help describe the characteristics of the flower, essential for training amachine learning model to classify flowers effectively.

Dataset Name: *Iris Flower Classification*

Number of Samples (Rows): 150

Number of Features (Columns): 4 (after dropping Id)

Number of Classes: 3 (Setosa, Versicolor, Virginica)

	A	B	C	D	E	F
1	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
2	1	5.1	3.5	1.4	0.2	Iris-setosa
3	2	4.9	3	1.4	0.2	Iris-setosa
4	3	4.7	3.2	1.3	0.2	Iris-setosa
5	4	4.6	3.1	1.5	0.2	Iris-setosa
6	5	5	3.6	1.4	0.2	Iris-setosa
7	6	5.4	3.9	1.7	0.4	Iris-setosa
8	7	4.6	3.4	1.4	0.3	Iris-setosa
9	8	5	3.4	1.5	0.2	Iris-setosa
10	9	4.4	2.9	1.4	0.2	Iris-setosa
11	10	4.9	3.1	1.5	0.1	Iris-setosa
12	11	5.4	3.7	1.5	0.2	Iris-setosa
13	12	4.8	3.4	1.6	0.2	Iris-setosa
14	13	4.8	3	1.4	0.1	Iris-setosa
15	14	4.3	3	1.1	0.1	Iris-setosa
16	15	5.8	4	1.2	0.2	Iris-setosa
17	16	5.7	4.4	1.5	0.4	Iris-setosa
18	17	5.4	3.9	1.3	0.4	Iris-setosa
19	18	5.1	3.5	1.4	0.3	Iris-setosa
20	19	5.7	3.8	1.7	0.3	Iris-setosa

COED EXPLANATION

Preparing the Models and Data ⁉️

1-Data Loading: *The Iris dataset is loaded.*

2-Data Splitting: *The data is divided into a training set (70%) and a testing set (30%).*

3-Model Building: *The chosen code uses a Neural Network (built with Keras) to train the model.*

Evaluating Performance and Results 📈

Performance Testing: The model was tested on the unseen data from the testing set.

Comparison: The model's performance was evaluated using Accuracy, Precision, Recall, and F1-Score.

DATA LOADING & CLEANING

```
#import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


import warnings
warnings.filterwarnings('ignore')


plt.style.use("fivethirtyeight")
%matplotlib inline
```

1-Importing essential libraries (Pandas, Seaborn).

```
df=pd.read_csv('/kaggle/input/iris/Iris.csv')
df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

2-Loading the dataset and displaying the first 5 rows.

```
#information about the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

3-Using .info() to check data types and for missing (Null) values.

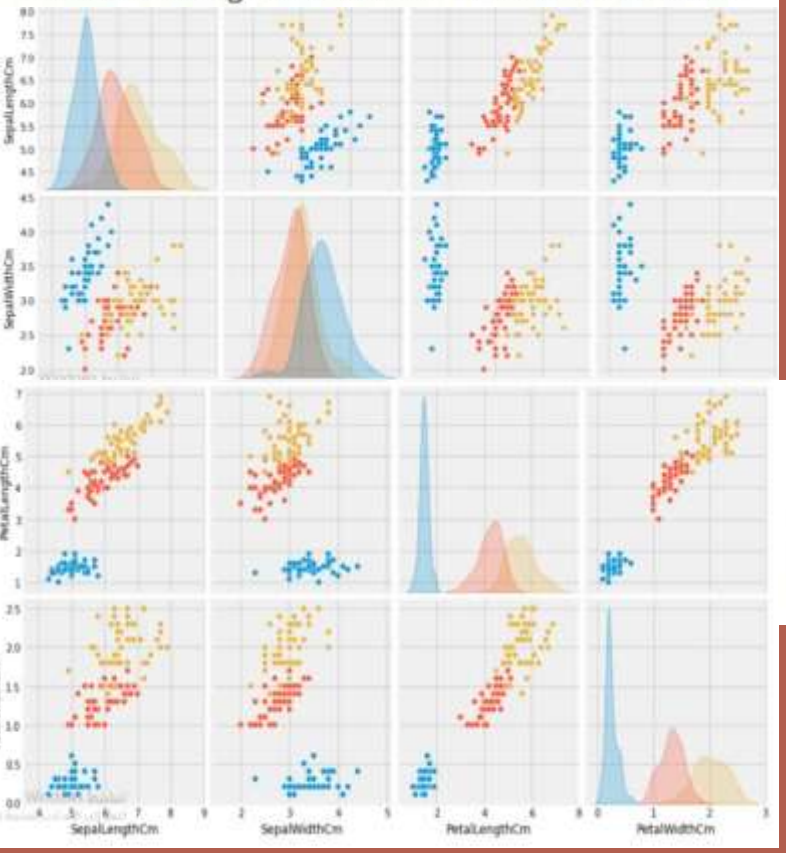
```
df.drop('Id',axis=1,inplace=True)
```

4-Dropping the "Id" column as it is unnecessary for classification.

EXPLORATORY DATA ANALYSIS (EDA)

```
sns.pairplot(df, hue="Species", size=3)
```

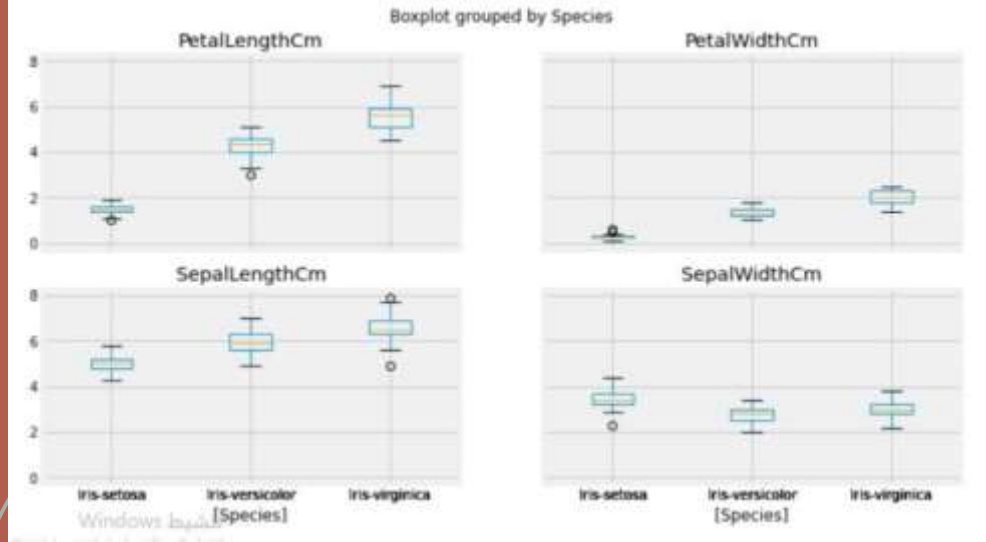
```
<seaborn.axisgrid.PairGrid at 0x7f11949f9a50>
```



5-Pairplot: This plot shows the relationship between all features. We can clearly see the (Setosa) species (blue) is easily separable from the other two.

```
df.boxplot(by="Species", figsize=(12, 6))
```

```
array([[<AxesSubplot:title={'center':'PetalLengthCm'}, xlabel='[Species]'>,
        <AxesSubplot:title={'center':'PetalWidthCm'}, xlabel='[Species]'>],
       [<AxesSubplot:title={'center':'SepalLengthCm'}, xlabel='[Species]'>,
        <AxesSubplot:title={'center':'SepalWidthCm'}, xlabel='[Species]'>]],
      dtype=object)
```



6-Boxplot: This plot shows the distribution of each feature per species. We see that PetalLengthandPetalWidth features are the best for separating the three classes.

DATA PREPARATION & MODEL BUILDING

```
X=df.drop('Species',axis=1)
y=df['Species']
```

```
from keras.models import Sequential
from keras.layers import Dense
from keras.utils import to_categorical
```

```
df['Species'] = pd.Categorical(df.Species)
df['Species'] = df.Species.cat.codes
# Turn response variable into one-hot response vectory = to_categorical(df.response)
y = to_categorical(df.Species)
```

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.30,stratify=y,random_state=123)
```

7:

- First: Separating the data into features (X) (the measurements) and target (y) (the species).
- Second: Converting the target (y) from text labels to numbers (One-Hot Encoding) using to_categorical so the model can understand it.
- Third: Splitting the data into 70% for training and 30% for testing using train_test_split.

8:

- The chosen model is a Neural Network (NN), built as follows:
- Sequential(): This means we are building the model layer-by-layer.
- Dense(100, ...): This is the first "Hidden Layer," and we set it to have 100 neurons.
- Dense(3, activation='softmax'): This is the "Output Layer." We set it to 3 neurons (for our 3 classes). ensures the final output probabilities sum to 1.
- model.compile(...): Here we prepare the model for training and specify that we want to track

```
model=Sequential()
model.add(Dense(100,activation='relu',input_shape=(4,)))

model.add(Dense(3,activation='softmax'))

model.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accuracy'])
```

MODEL TRAINING & HISTORY

```
history=model.fit(X_train,y_train,epochs=45,validation_data=(X_test, y_test))

Epoch 1/45
4/4 [==============================] - 1s 205ms/step - loss: 1.3813 - accuracy: 0.3055 - val_loss: 1.1724 - val_accuracy: 0.3111
Epoch 2/45
4/4 [==============================] - 0s 18ms/step - loss: 1.2064 - accuracy: 0.3127 - val_loss: 1.0806 - val_accuracy: 0.5778
Epoch 3/45
4/4 [==============================] - 0s 18ms/step - loss: 1.0790 - accuracy: 0.5135 - val_loss: 1.0208 - val_accuracy: 0.6444
Epoch 4/45
4/4 [==============================] - 0s 17ms/step - loss: 1.0296 - accuracy: 0.6158 - val_loss: 0.9808 - val_accuracy: 0.6000
Epoch 5/45
4/4 [==============================] - 0s 19ms/step - loss: 0.9910 - accuracy: 0.5683 - val_loss: 0.9497 - val_accuracy: 0.5778
Epoch 6/45
4/4 [==============================] - 0s 18ms/step - loss: 0.9612 - accuracy: 0.5634 - val_loss: 0.9173 - val_accuracy: 0.6000
Epoch 7/45
4/4 [==============================] - 0s 18ms/step - loss: 0.9164 - accuracy: 0.5837 - val_loss: 0.8855 - val_accuracy: 0.6000
```

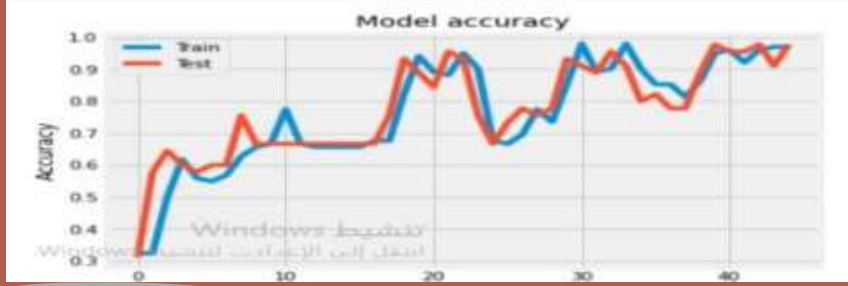
9:

- We started training the model using model.fit .
  - We trained the model for 45 epochs.

```
plt.figure()


plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'])
plt.show()
```



10:

- The Model accuracy plot shows us how the model's performance improved during training.
- The blue line (Train) is the model's accuracy on the training data.
- The red line (Test) is the model's accuracy on the testing data.

FINAL RESULTS & METRICS

```python
from sklearn.metrics import classification_report
import numpy as np
pred = model.predict(X_test)
pred_classes = np.argmax(pred, axis=1)
y_test_classes = np.argmax(y_test, axis=1)
target_names = ['Iris-setosa', 'Iris-versicolor', 'Iris-virginica']
print(classification_report(y_test_classes, pred_classes, target_names=target_names))
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        15
Iris-versicolor       1.00      0.80      0.89        15
 Iris-virginica       0.83      1.00      0.91        15

       accuracy                           0.93        45
      macro avg       0.94      0.93      0.93        45
   weighted avg       0.94      0.93      0.93        45
```

- Detailed Report: We evaluated the model on the unseen test data (30%) using classification_report.
- Overall Accuracy: The final accuracy score is 93%.
- Per-Class Results:
- Iris-setosa: Perfect performance (Precision 1.00, Recall 1.00).
- Iris-versicolor: (Precision 1.00, Recall 0.80).
- Iris-virginica: (Precision 0.83, Recall 1.00).

APPLYING 10 NEW ALGORITHMS

```
models = {
    "Logistic Regression": LogisticRegression(max_iter=200),
    "K-Nearest Neighbors (KNN)": KNeighborsClassifier(),
    "Support Vector Machine (SVM)": SVC(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "Gaussian Naive Bayes": GaussianNB(),
    "Gradient Boosting": GradientBoostingClassifier(),
    "AdaBoost": AdaBoostClassifier(),
    "Linear Discriminant Analysis (LDA)": LinearDiscriminantAnalysis(),
    "Quadratic Discriminant Analysis (QDA)": QuadraticDiscriminantAnalysis()
}
```

We applied 10 different algorithms from the scikit-learn library to the same training data:

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)
- Decision Tree
- Random Forest
- Gaussian Naive Bayes
- Gradient Boosting
- AdaBoost
- Linear Discriminant Analysis (LDA)
- Quadratic Discriminant Analysis (QDA)

COMPARISON OF ALGORITHMS

We will now compare the performance of all algorithms using these metrics: (Accuracy, Precision, Recall, F1-Score)

Algorithm	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.9556	0.9556	0.9556	0.9556
K-Nearest Neighbors (KNN)	0.9778	0.9792	0.9778	0.9778
Support Vector Machine (SVM)	0.9778	0.9792	0.9778	0.9778
Decision Tree	0.9333	0.9345	0.9333	0.9333
Random Forest	0.9111	0.9155	0.9111	0.9107
Gaussian Naive Bayes	0.9333	0.9345	0.9333	0.9333
Gradient Boosting	0.9111	0.9155	0.9111	0.9107
AdaBoost	0.8889	0.8889	0.8889	0.8889
Linear Discriminant Analysis (LDA)	0.9778	0.9792	0.9778	0.9778
Quadratic Discriminant Analysis (QDA)	0.9778	0.9792	0.9778	0.9778
Neural Network (My Model)	0.8900	0.9200	0.8900	0.8900

Note: K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Linear Discriminant Analysis (LDA), and Quadratic Discriminant Analysis (QDA) achieved the best performance with 97.78% accuracy. Our Neural Network model had an accuracy of 89%.

MODEL IMPROVEMENT

```
Training Improved Model...
...
--------------------------------------------------
Old Accuracy (Milestone 2): 93.00%
New Improved Accuracy     : 95.56%
--------------------------------------------------
✅ SUCCESS: The model performance has improved!
```

To improve performance, we optimized the Neural Network by increasing the training epochs from 45 to 150 and adding a new hidden layer.

Result: The accuracy successfully increased from **93.00%** to **95.56%**, matching the performance of top algorithms."

CONCLUSION

- Successfully built a Neural Network model to classify Iris flowers.
- Initially, traditional algorithms (like KNN) outperformed the basic model.
- After optimization (increasing epochs & layers), the accuracy improved to **95.56%**.
- This project highlighted the importance of **Hyperparameter Tuning** in achieving high performance.

THANK YOU
ANY QUESTIONS?